Levi Honigford, Kyle Russel, Aaron Pinnell, Kevin Chu
4/23/23
CS 1021
Project Phase 2 Report

I. Classes

User

Data

- int SSN
- string Username
- string Name
- string Organization
- string Password
- date Date_Of_Birth
- date Date_Of_Join

Admin

Data

- int AID
- int Rank

Functions

- Check_Username(string Username)
- Check_Password()
- Display_Name()
- getRank()
- getAID()
- Display_Info()
- Create_Admin()
- Admin_Delete(int ID)

Customer

Data

- int CID
- int Strikes
- string Account_Type

- string Status
- vector<string> Messages
- vector<float> Transactions
- vector<string> Transaction_Types
- float Current_Balance

Functions

- New_Message(string Message)
- New_Transaction(float Transaction, string Transaction_Type)
- Pay_Interest()
- Increase_Balance(float Inc)
- Check_Username(string Username)
- Check_Password()
- Strike_Check()
- Display_Name()
- Display_Info()
- Positive_Display()
- Negative_Display()
- Positive_Check()
- Active_Check()
- Block_Account()
- Statement_Summary()
- Display_Balance()
- Withdraw()
- Deposit()
- Transfer()

II. Interface

Preloading Phase

This phase is essentially in charge of loading in user data from the data text files to allow for users to sign in. The flow of code is as follows:

- Checks for the existence of a creator admin. If one doesn't exist, it grants the user temporary administrative permissions to create a creator admin account via the admin signup page.
- Uses fstream library to open the data text files and decrypts them using the decryption function.

- Uses a for loop to assign all the user data to their respective vectors based on their account type (Admin or Customer).

Main Page

This main page comes after the preloading phase and presents the user with several options including signing into accounts or signing up for an account. The flow of code is as follows:

- Extra checks for the existence of a creator admin. If one doesn't exist, it grants the user temporary administrative permissions to create a creator admin account via the admin signup page.
- Runs a while loop to output a simple user interface message that prompts the user for an input based on values 1-4. This while loop keeps the code from ending if an invalid input is made.
- The options given were to login to an admin account, login to a customer account, sign up for a customer account, and to close the application.
- Based on the value of the user input, it would redirect them to their corresponding pages by executing their respective functions for page redirection.

Admin Creation Page

This page is exclusive for the creator admin in the regard that it is essentially where other admin accounts are created. The flow of code is as follows:

- Prompts the user for the new admin's information (Username, Name, Organization, SSN, DOB, Password)
- One important informational prompt to mention is the addition of admin ranks, in which depending on the admin rank directly affects what actions they have access to.
- Pushes this new account into an admin data vector.

Admin Sign-in Page

This page is essentially in charge of making sure the user inputs the correct username and password to an admin account to sign them into their personal home pages. The flow of code is as follows:

- Prompts the user for a valid username and the correct corresponding password.
- If both username and password check out, redirect to the personalized admin home page via their admin account's ID.

Customer Sign-up Page

This page oversees guiding new users to create their own customer account. The flow of code is as follows:

- Creates a new account for the user using the customer class.

- Prompts the user for the new customer's information (Username, Name, Account Type, Organization, SSN, DOB, Password)
- Pushes the new account into a customer data vector.

Customer Sign-in Page

This page is essentially in charge of making sure the user inputs the correct username and password to a customer account to sign them into their personal home pages. The flow of code is as follows:

- Prompts the user for a valid username and the correct corresponding password.
- If both username and password check out, redirect to the personalized customer home page via their customer account's ID.

Admin Home Page

This page presents the actions that the admin account can do based on their admin rank. The flow of code is as follows:

- Syncs the home page to their admin user ID.
- Runs a while loop to output a simple user interface message that prompts the user for an input based on values 1-9 based on their admin rank (Admins with rank 3 will only see actions at and under the 3rd option). This while loop keeps the code from ending if an invalid input is made.
- The actions included are:
    1) Logout of the Account
    2) Check accounts with positive balance
    3) Check accounts with negative balance
    4) Generate a summary Report
    5) Send warning messages to negative accounts
    6) Pay monthly interest to accounts
    7) Block accounts with repeated negative balance
    8) Create new admin
    9) Delete admin
- Each input will run their corresponding function

Customer Home Page

This page presents the actions that the customer account can do. The flow of code is as follows:

- Syncs the home page to their customer user ID.
- Runs a while loop to output a simple user interface message that prompts the user for an input based on values 1-7. This while loop keeps the code from ending if an invalid input is made.

- The actions included are:
    1) Statement summary
    2) Check balance
    3) Withdraw
    4) Deposit
    5) Transfer to other account
    6) Display messages
    7) Logout and save
- Each input will run their corresponding function

III. Functions

clr()

- A void function that flushes the output stream and clears the terminal screen.

pause()

- A void function that outputs a message to the user that page change is occurring and receives any input to continue with the clearing and printing.

isInt(string x)

- Bool function that checks if a string is an integer value.

isFloat(string x)

- Bool function that checks if a string is a float value.

string_to_vector(string x)

- A vector<string> function that converts a single string taken from the fstream into a vector of strings based on semi-colons.

Transaction_Convert(vector<string>)

- A vector<float> function that converts a vector of strings into float values to take the fstream inputs and turn them into the proper transaction vector for storage.

Currencystof(float x)

- A string function that takes a float value and turns it into a string and then takes off the last 4 characters, making the decimal only 2 numbers, the form of currency representation.

dash_to_space(string x)

- A string function that converts all the dashes in a string into spaces, used to make sure fstream doesn't read spaces in messages as spaces in data storage.

space_to_dash(string x)

- A string function that converts all the spaces in a string into dashes to be stored in the fstream.

encryption(string x)

- A string function that applies a Caeser cipher to shift all characters ASCII a certain amount when writing to the fstream.

decryption(string x)

- A string function that shifts all the ASCII characters back down the same amount the encryption function shifts them up.

localtime_xp(time_t timer)

- A function using machine time to get the current date in DD/MM/YYYY form.

Date_Check(date DOB, date DOJ)

- A bool function used to check if a date of birth is 18 years old based on current date.

Save_Data()

- A void function that encrypts all the data and the writes it to the respective data file

Customer_Signup()

- A void function that directs to customer signup page (see II. Interface)

Customer_Home(int ID)

- A void function that directs to customer home page (see II. Interface)

Customer_Login()

- A void function that directs to customer Log in page (see II. Interface)

Admin_Home()

- A void function that directs to Admin home page (see II. Interface)

Admin_Login()

- A void function that directs to Admin Log in page (see II. Interface)